DEPARTMENT OF COMPUTER SCIENCE / INFORMATION TECHNOLOGY

CyberShield

A Python-Based Cybersecurity Monitoring Dashboard

Mini Project Report submitted in partial fulfilment of the course requirements

Student Name: _____

Roll Number: _____

Course / Semester: _____

Guide / Faculty: _____

Academic Year: 2024 – 2025

Date of Submission: _____

Abstract

CyberShield is a small web-based security dashboard built with Python, Streamlit and Pandas. It shows login activity as number cards, charts, alerts and a filterable table. A simple rule-based check flags any IP address with five or more failed login attempts as suspicious, assigns it a risk level (Low, Medium or High) and explains why it was flagged. The project uses only fictional sample data and does not connect to any real system. It is designed to be easy to read and understand, and it shows the basic idea behind login monitoring without using machine learning or complex infrastructure.

1. Introduction

Every system that asks users to log in produces a record of those attempts. Most attempts are normal, such as a user typing the right password or making a small typo. Sometimes, however, one source makes many failed attempts in a short time. This can be a sign that someone is guessing passwords, although it can also have harmless causes.

Security teams use dashboards to see this kind of activity at a glance. CyberShield is a beginner-friendly version of such a dashboard. It focuses on clarity: a few key numbers, two simple charts, a clear warning when something looks unusual, and a table of the

underlying records.

2. Problem Statement

Login records are long lists of rows, and it is hard to notice a suspicious pattern by reading them one by one. A person needs a quick way to answer questions such as:

- How many login attempts were made, and how many failed?
- Is any single IP address failing again and again?
- How serious does that pattern look, and why was it flagged?

CyberShield answers these questions on one page using a simple, explainable rule.

3. Objectives

1. Build a clean, modern, dark-themed security dashboard using Python and Streamlit.
2. Display four key numbers: total logins, successful logins, failed logins and suspicious activities.
3. Generate fictional login records that include both normal logins and a suspicious pattern.
4. Detect suspicious IP addresses with a simple, easy-to-explain rule.
5. Present alerts with a risk level and a clear reason.
6. Visualise the data with two simple charts and provide filters to explore it.
7. Keep all code in one file (`app.py`) and keep it simple enough for beginners.

4. Scope and Limitations

In scope

- A single-page dashboard running locally in the browser.
- Twenty fictional login records created when the user clicks a button.
- One detection rule based on the number of failed logins per IP address.
- Risk labels, alerts, two charts and two dropdown filters.

Out of scope

- Connecting to real systems, servers, logs or networks.
- Machine learning or any automatic learning from data.

- User accounts, a database, or saving data between sessions.
- Blocking IP addresses or taking any automatic action.

Limitations

- The data is fictional and small, so results show the idea rather than real-world performance.
- The rule counts failures only. It does not consider timing, location or successful logins that follow failures.
- Many users can share one IP address (for example, in an office), so a flagged IP is a clue and not proof of an attack.

4.1 Ethical Note

A suspicious login pattern does not automatically prove a cyberattack. Harmless explanations include a forgotten password, a misconfigured application, or many people sharing one network address. For this reason CyberShield describes its results as alerts to review, and it shows this reminder on the dashboard itself. All data and IP addresses in this project are fictional. The IP addresses come from ranges reserved for documentation and examples, so they do not belong to real devices.

5. Technologies Used

Technology	Purpose
Python 3	Main programming language
Streamlit	Builds the web page, button, dropdown filters and layout
Pandas	Stores the login records in a table, counts and filters them
HTML and CSS (inside Streamlit)	Dark theme, number cards, risk labels, bar charts and tables

The whole project is a single file named `app.py`. It can be started with the command `streamlit run app.py`.

6. System Design

6.1 Overall flow

1. The user opens the dashboard. No data is shown yet, only empty-state messages.
2. The user clicks **Generate Sample Data**. Twenty fictional records are created and stored in Streamlit's `session_state`, so they are remembered when the page refreshes after a filter change.
3. The program counts total, successful and failed logins.
4. The detection rule runs over the records and produces a list of suspicious IP addresses.
5. The page shows the cards, charts, warnings, alerts table and login records table.
6. When the user changes a dropdown, the page refreshes and the affected table is filtered.

6.2 Data structure

Each login record has four fields:

Field	Example
Time	2026-10-21 09:20:47
Username	admin
IP Address	203.0.113.45
Result	Failed

The records are held in a Pandas table (DataFrame) with these four columns.

6.3 Main parts of the code

Part	What it does
`make_sample_data()`	Creates the 20 fictional login records
`find_suspicious_ips()`	Applies the security rule and returns the flagged IP addresses

Part	What it does
`get_risk_level()`	Converts a failed-attempt count into Low, Medium or High
`build_alerts_table()`	Builds the Security Alerts table with coloured risk labels
`build_bar_chart()`	Draws a simple bar chart from a list of labels and numbers
`show_card()`	Draws one number card

7. Implementation

7.1 Dashboard layout and dark theme

The page uses a dark background with a blue title and four cards in one row. Each card has its own colour: blue for total logins, green for successful logins, orange for failed logins and red for suspicious activities. A short block of CSS sets the colours, spacing and rounded corners. The tables and charts are drawn as HTML so that they stay readable in the dark theme.

7.2 Fictional sample data

The sample contains 20 records created by a button click. It includes normal successful logins by users such as alice, bob and carol. It also includes harmless mistakes, such as a user who mistypes a password once and then logs in correctly. Finally, it includes six failed attempts from the single IP address `203.0.113.45`, trying the usernames admin, root, guest and test within about 40 seconds. This is the pattern the rule is designed to find.

7.3 Detection rule

The rule is:

> If the same IP address has **five or more** failed login attempts, mark it as suspicious.

The program works in two steps using plain loops and `if` statements:

1. **Count.** Go through every record. For each failed login, add 1 to the counter for that IP address and remember which usernames it tried.

2. **Check.** Go through the counters. Any IP address whose count is at or above the limit (`FAILED_LIMIT = 5`) is added to the suspicious list.

The number on the Suspicious Activities card is the number of flagged IP addresses, and it updates automatically whenever the data changes. The limit is stored in one named value, so it is easy to change.

7.4 Risk levels

Each flagged IP address receives a risk label based on how many failed attempts it made:

Failed attempts	Risk level	Colour
5	Low	Green
6 to 9	Medium	Orange
10 or more	High	Red

The two boundaries are stored as `MEDIUM_RISK_LIMIT = 6` and `HIGH_RISK_LIMIT = 10`.

7.5 Security Alerts section

For each flagged IP address the dashboard shows a red warning box and a row in the Security Alerts table. The table lists the suspicious IP address, the usernames it tried, the number of failed attempts, a coloured risk label and a short reason, for example: "6 failed logins from this IP (limit is 5), trying 4 different username(s)." A note under the table reminds the reader that a suspicious pattern is not proof of an attack.

7.6 Charts

The dashboard has two simple bar charts. Pandas does the counting and the bars are drawn with HTML and CSS.

1. **Successful vs Failed Logins.** Two bars compare the number of successful logins with the number of failed logins. This shows what share of login traffic is failing. Some failures are normal, but a sudden jump is worth a closer look.
2. **Failed Login Attempts by IP Address.** One bar per IP address shows how many failures came from it. Bars for IP addresses flagged by the rule are red and the others are orange. This shows whether failures are spread out or concentrated in one place, so it points to where a problem may be coming from.

Together the charts tell a story. The first says whether there is a problem, and the second says where it is coming from.

7.7 Filters

Two dropdown filters let the user explore the data:

- **Filter alerts by risk level** (All, High, Medium, Low) changes which rows appear in the Security Alerts table. If no alert matches, a clear message is shown.
- **Filter by login result** (All, Success, Failed) changes which rows appear in the login records table, with a line such as "Showing 8 of 20 records."

The cards and charts always use the full data, so the overall picture stays consistent.

8. Testing and Results

The detection and counting logic was tested in Python on the sample data. The dashboard itself should be checked by running it and comparing what appears on screen with the expected results below.

Test	Expected result
Open the page before clicking the button	Cards show 0, and empty-state messages appear for charts, alerts and records
Click Generate Sample Data	20 records appear; Total 20, Successful 12, Failed 8
Check the suspicious count	Suspicious Activities shows 1 (IP `203.0.113.45`)
Check the alert	Risk level Medium, 6 failed attempts, usernames admin, root, guest, test
Check that harmless failures are not flagged	The IPs with one failed login each are not flagged
Check a case below the limit (only four failures from one IP)	The IP is not flagged
Filter alerts by High or Low	The message "No alerts with ... risk level" appears

Test	Expected result
Filter alerts by Medium	The alert is shown
Filter records by Failed	8 of 20 records are shown
Filter records by Success	12 of 20 records are shown
Set `HIGH_RISK_LIMIT = 6` and refresh	The alert becomes High and appears under the High filter

[Insert screenshot: dashboard before generating data]

[Insert screenshot: dashboard after generating data, showing cards, charts and alert]

[Insert screenshot: filters in use]

Summary of results

With the sample data, the dashboard reports 20 total logins, 12 successful, 8 failed and 1 suspicious activity. The flagged IP address has 6 failed attempts, receives a Medium risk label and is shown in red in the second chart. The harmless failures are correctly left unflagged.

9. Advantages

- Very easy to read and understand, even for beginners.
- Every alert comes with a plain-language reason, so the rule is explainable.
- Uses only fictional data and is safe to run and demonstrate.
- Small and simple: one file, a few functions and no complex setup.
- Thresholds are single values that are easy to adjust.

10. Future Scope

- Read login records from a CSV file or log file instead of using built-in sample data.
- Add time-based rules, such as five failures within one minute.
- Highlight the flagged IP's rows in the login records table.
- Add more rules, for example many different usernames from one IP or a success right after many failures.
- Add a time chart of failed logins and an export option for alerts.

- Add user sign-in and a stored history of alerts.

11. Conclusion

CyberShield demonstrates the core idea of login monitoring in a simple form. It counts login activity, applies an easy-to-explain rule to find IP addresses with many failed attempts, labels each alert with a risk level and a reason, and presents everything in a clear dark-themed dashboard with charts and filters. The project shows how Python, Streamlit and Pandas can be combined to build a useful and understandable security tool. It also stresses an important point: a suspicious pattern is a reason to investigate, not proof of an attack.

12. How to Run the Project

1. Install Python 3.9 or newer.
2. Install Streamlit (Pandas is installed with it): `pip install streamlit`
3. Save the code as `app.py` and open a terminal in the folder that contains it.
4. Start the dashboard: `streamlit run app.py` (or `python -m streamlit run app.py`).
5. Open `http://localhost:8501` in the browser if it does not open automatically.
6. Click **Generate Sample Data** to load the records.

References

1. Streamlit Documentation. https://docs.streamlit.io
2. Pandas Documentation. https://pandas.pydata.org/docs
3. Python Documentation. https://docs.python.org/3